

SmartRoom Manager

Réservation intelligente de salles de campus — de l'analyse du besoin à la mise en production, en passant par la conception, la réalisation, la qualité logicielle et l'exploitation.

Nature	Rapport de projet
Formation	Bachelor 3 Data & Intelligence Artificielle
Établissement	ECE Paris
Auteur	Dylan Menga Wanda
Application	https://smartroommanager.vercel.app
Dépôt	https://github.com/DD542/SmartRoomManager

Sommaire

1. Contexte, problème et périmètre

- 1.1 La situation observée
 - 1.2 Les trois problèmes traités
 - 1.3 Ce que le projet livre
 - 1.4 Cadre académique et dimension Data & IA
 - 1.5 Méthode de travail
-

2. Analyse

- 2.1 Acteurs
 - 2.2 Cas d'usage
 - 2.3 Exigences fonctionnelles
 - 2.4 Exigences non fonctionnelles
 - 2.5 Contraintes et leurs conséquences
-

3. Conception fonctionnelle et maquettage Figma

- 3.1 Démarche de maquettage
 - 3.2 Inventaire des écrans
 - 3.3 Système de design
 - 3.4 Le tunnel de réservation
 - 3.5 Du maquettage au code
-

4. Architecture technique

- 4.1 Vue d'ensemble
 - 4.2 Découpage en couches
 - 4.3 Justification des choix
 - 4.4 Le flux de l'assistant
-

5. Modèle de données

- 5.1 Volumétrie et domaines
 - 5.2 La contrainte qui porte l'intégrité
 - 5.3 Recherche vectorielle
 - 5.4 Stratégie de migration
-

6. Réalisation — back-end

- 6.1 Authentification et sessions

- 6.2 Permissions
 - 6.3 Concurrence et interblocages
 - 6.4 Ordonnanceur
 - 6.5 Moteur de recommandation
 - 6.6 Notifications et courriels
-

7. Réalisation — front-end

- 7.1 Organisation
 - 7.2 Couche d'accès et adaptateurs
 - 7.3 Gardes de routage
 - 7.4 Un piège de fuseau horaire
-

8. L'assistant conversationnel

- 8.1 Catalogue d'outils
 - 8.2 Architecture à trois étages
 - 8.3 Un repli parfois choisi
 - 8.4 Recherche documentaire hybride
 - 8.5 Garde-fous
 - 8.6 Budget et observabilité
-

9. Sécurité et conformité

- 9.1 Gestion des secrets
 - 9.2 Cloisonnement des données personnelles
 - 9.3 Retrait d'un compte par anonymisation
 - 9.4 Journal d'audit
-

10. Stratégie de tests

- 10.1 Cinq niveaux, cinq intentions
 - 10.2 Chiffres
 - 10.3 Le principe de la contre-épreuve
 - 10.4 Ce que les tests ont réellement attrapé
-

11. Intégration continue

- 11.1 Composition de la chaîne
 - 11.2 Trois causes d'échec indépendantes
 - 11.3 Deux pièges d'environnement
-

12. Fonctionnement en local

- 12.1 Démarrage de la pile
 - 12.2 Le préchauffage des modèles
 - 12.3 Le budget de premier jeton
 - 12.4 Jeu de démonstration
-

13. Déploiement en production

- 13.1 Cible retenue
 - 13.2 Le front devant l'API
 - 13.3 Point d'entrée du conteneur
 - 13.4 La sonde de démarrage
 - 13.5 Chaîne d'incidents de connexion
 - 13.6 Persistance des médias
 - 13.7 L'assistant en ligne
 - 13.8 Supervision
-

14. Journal complet des incidents

15. Écarts entre local et production

16. Limites connues et perspectives

17. Bilan et enseignements

18. Annexes

1 Contexte, problème et périmètre

1.1 La situation observée

Un campus dispose de salles de natures très différentes — amphithéâtres de quatre-vingt-dix places, laboratoires équipés, salles de réunion de quatre à douze personnes — réparties dans plusieurs bâtiments parfois distants de plusieurs rues. Leur réservation se fait couramment par un tableur partagé, un courriel au secrétariat, ou un affichage papier collé sur la porte.

Ces trois mécanismes ont un point commun : **aucun ne peut garantir qu'une salle n'est pas déjà prise**. Le tableur autorise deux écritures simultanées. Le courriel dépend de la disponibilité d'une personne. Le papier n'est visible que par celui qui passe devant.

1.2 Les trois problèmes traités

ON NE SAIT PAS CE QUI EST LIBRE

Chercher une salle revient à ouvrir plusieurs sources et à les recouper mentalement. Le coût n'est pas le temps passé : c'est le renoncement. La personne réserve « au cas où », ce qui bloque un créneau dont elle ne se servira peut-être pas, et aggrave la pénurie qu'elle subissait.

DEUX PERSONNES RÉSERVENT LE MÊME CRÉNEAU

Sans arbitre technique, le conflit ne se découvre pas au moment de la réservation mais devant la porte. Le coût réel n'est pas la double écriture en base : c'est la réunion qui n'a pas lieu, et les six personnes qui repartent.

LES SALLES RÉSERVÉES RESTENT VIDES

Personne ne vient, et le créneau reste bloqué pour tous les autres. Un système qui ne mesure pas la présence effective ne peut pas libérer ce qui ne sert pas. C'est le problème le moins visible et le plus coûteux, parce qu'il est invisible : la salle apparaît occupée dans le système et vide dans la réalité.

1.3 Ce que le projet livre

Une application web complète — interface utilisateur, interface d'administration, API documentée, base de données, ordonnanceur et assistant conversationnel — qui répond aux trois problèmes par trois mécanismes distincts :

- **Trouver** — une recherche multicritère et un moteur de recommandation qui n'affichent que des salles réellement libres et conformes aux règles en vigueur.
- **Réserver** — une écriture atomique dont la garantie de non-chevauchement est portée par la base de données elle-même, et non par du code applicatif qu'on pourrait contourner.
- **Entrer et libérer** — un code d'accès émis à l'approche du créneau, une validation de présence dans une fenêtre courte, et une libération automatique du créneau lorsque personne ne se présente.

1.4 Cadre académique et dimension Data & IA

Projet individuel de **Bachelor 3 Data & Intelligence Artificielle** à l'ECE Paris. L'application n'est pas exploitée commercialement et ne constitue pas un service ouvert au public — ce que rappellent explicitement ses mentions légales.

La dimension Data & IA n'est pas un habillage ajouté après coup : elle porte deux briques fonctionnelles réelles du produit.

Le **moteur de recommandation** ordonne les salles compatibles selon un score composite dont chaque composante est calculée séparément et exposée. L'interface peut donc dire *pourquoi* une salle arrive en tête — exigence d'explicabilité qui a orienté toute sa conception.

L'**assistant conversationnel** répond en langage naturel en s'appuyant sur quatorze outils qui interrogent la vraie base, et sur une base de connaissances vectorisée interrogée par recherche hybride. Il est encadré par trois garde-fous : détection d'injection, contrôle d'étayage, et anonymisation avant tout envoi à un tiers.

1.5 Méthode de travail

Une règle a structuré tout le projet, et particulièrement sa seconde moitié : **mesurer avant d'affirmer**. Plusieurs diagnostics initiaux se sont révélés faux, et à chaque fois c'est une mesure — souvent qu'il a d'abord fallu rendre possible — qui a tranché.

Cette discipline a une conséquence visible dans le dépôt : les messages de commit ne décrivent pas seulement ce qui change, mais ce qui a été observé, ce qui a été essayé et abandonné, et ce qui prouve que la correction fonctionne.

PRINCIPE RETENU

Un test qui ne peut pas échouer ne prouve rien. Chaque correction d'importance est accompagnée d'une contre-épreuve : on vérifie que le test échoue bien lorsqu'on retire le correctif. Cette pratique a, à elle seule, invalidé un correctif qui semblait bon — le motif de recherche avait été replié sur plusieurs lignes par le formateur, et le remplacement n'avait rien changé.

2 Analyse

2.1 Acteurs

Acteur	Ce qu'il fait	Ce qu'il ne peut pas faire
Utilisateur	Cherche une salle, réserve, modifie, annule, valide sa présence, consulte ses statistiques, ouvre un ticket, dialogue avec l'assistant	Voir les données d'autrui, modifier le parc, changer les règles
Administrateur	Gère le parc, arbitre les conflits, configure les règles, traite le support, exporte les données	Agir hors de son périmètre de permissions
Propriétaire	Administrateur portant les sept permissions, gère les autres administrateurs et leurs invitations	—
Ordonnanceur	Libère les créneaux sans présence, clôt les réservations passées, envoie rappels et courriels, rafraîchit les agrégats, purge les jetons	—

POURQUOI SÉPARER ADMINISTRATEUR ET PROPRIÉTAIRE

Une matrice de permissions où tout le monde a tout ne prouve rien, et l'écran des rôles n'aurait rien à montrer. Le jeu de démonstration porte donc **cinq comptes d'administration aux périmètres volontairement différents** : un propriétaire, un directeur de site limité aux salles, au support et aux conflits, une référente support limitée au support et aux conflits, et deux périmètres partiels. La démonstration du cloisonnement devient alors possible devant un jury.

2.2 Cas d'usage

Réf.	Cas d'usage	Déroulé et règles appliquées
U1	Réserver une salle	L'utilisateur décrit son besoin (date, plage, effectif, équipements). Le système ne propose que des salles libres et conformes. Confirmation par courriel ; code d'accès une heure avant le créneau.
U2	Valider sa présence	À l'ouverture du créneau, saisie du code affiché sur l'écran de la salle. Fenêtre de dix minutes ; passé ce délai, le créneau est libéré automatiquement.
U3	Annuler ou modifier	Jusqu'à une heure avant le début, délai configurable par salle. Motif obligatoire. Les participants sont prévenus par courriel.
U4	Demander un accès	Certaines salles exigent un badge. L'utilisateur formule une demande motivée, un administrateur arbitre avec trace.
U5	Arbitrer un conflit	Deux demandes concurrentes sur un créneau contraint remontent à l'administration, sous une référence lisible (#CONF-8492).
U6	Configurer les règles	Durées minimale et maximale, battement, préavis, horizon, quota hebdomadaire, délai d'annulation, fenêtre de validation. Portée établissement, bâtiment ou salle.
U7	Interroger l'assistant	En langage naturel : ses réservations, une salle libre, l'emplacement d'une salle, une procédure, l'ouverture d'un ticket.
U8	Administrer le parc	Bâtiments, étages, plans d'étage, salles, photos, repères de localisation, équipements, statuts.
U9	Suivre l'occupation	Tableaux de bord, heures de pointe, taux d'occupation par salle, exports CSV.
U10	Gérer les comptes	Suspension motivée avec courriel, réactivation avec courriel, retrait définitif par anonymisation.

2.3 Exigences fonctionnelles

- Recherche multicritère : date, plage horaire, capacité, équipements, accessibilité PMR, bâtiment.
- Recommandation ordonnée, avec explication du score attribué.
- Réservation atomique, sans chevauchement possible en aucune circonstance.
- Réservations récurrentes, avec possibilité d'annuler une occurrence isolée sans toucher à la série.
- Participants, invitations et réponses.
- Code d'accès à usage unique par réservation, émis à l'approche du créneau.
- Validation de présence dans une fenêtre courte et bornée.
- Libération automatique du créneau en cas d'absence constatée.
- Fermetures exceptionnelles à trois portées, et horaires d'ouverture par bâtiment.
- Notifications applicatives et courriels, rendus depuis des gabarits modifiables sans redéploiement.
- Journal d'audit de toute décision d'administration, exportable.
- Statistiques personnelles et globales, exportables.

- Assistant conversationnel outillé, citant ses sources.

2.4 Exigences non fonctionnelles

Exigence	Traduction technique retenue
Intégrité	Contrainte <code>EXCLUDE USING gist</code> en base : aucun chevauchement n'est représentable, quel que soit le chemin d'écriture.
Confidentialité	Portées de session distinctes, permissions granulaires, en-têtes de cache stricts sur toute route personnelle.
Traçabilité	Journal d'audit métier, journal par tour d'assistant, journaux applicatifs structurés en JSON.
Robustesse	Dégradations prévues <i>et testées</i> : sans modèle de langage, sans modèle de vecteurs, sans relais de courriel, l'application continue de fonctionner.
Testabilité	Couche métier pure, sans dépendance, exigée à 100 % de couverture de branches.
Portabilité	Conteneurisation, migrations versionnées, aucune valeur de configuration codée en dur.
Accessibilité	Libellés visibles <i>et</i> lisibles par lecteur d'écran ; conformité WCAG 2.5.3 sur les déclencheurs.

2.5 Contraintes et leurs conséquences

Budget nul. L'hébergement devait tenir sur des paliers gratuits. Cette contrainte n'a pas été subie mais intégrée à la conception : elle a directement produit l'architecture à trois étages de l'assistant, la stratégie de versionnement des médias, et la dégradation propre décrite au chapitre 13.

Un seul développeur. D'où le choix d'outils à faible coût d'exploitation, et d'une chaîne d'intégration continue exigeante qui remplace la relecture par un pair : c'est elle qui a détecté trois défauts que la relecture aurait pu laisser passer.

Pas de GPU en production. Aucun hébergement gratuit ne fait tourner un modèle de sept milliards de paramètres. L'assistant devait donc fonctionner sans modèle local — ce qui a imposé une architecture où le modèle est un *confort* et non une *dépendance*.

3 Conception fonctionnelle et maquettage Figma

3.1 Démarche de maquettage

L'ensemble des écrans a été maqueté sur **Figma** avant toute ligne de code. Ce choix n'était pas cosmétique : le maquettage a servi trois usages, dont deux n'ont rien à voir avec l'apparence.

FIGER LE VOCABULAIRE

Les maquettes ont imposé un vocabulaire unique avant l'implémentation — *créneau*, *battement*, *préavis*, *horizon*, *repère*, *plan d'étage*. Sans cette étape, un même objet aurait porté trois noms entre l'interface, l'API et la base de données, comme cela arrive régulièrement lorsque l'écran est dessiné après le schéma.

La distinction entre **repère de localisation** (une image attachée à la salle, qui montre où elle se trouve) et **plan d'étage** (un document déposé par l'administration, sur lequel les salles sont positionnées) vient directement du maquettage. Elle a plus tard produit un vrai défaut, décrit à l'incident n° 12, précisément parce que l'implémentation avait oublié l'un des deux côté utilisateur.

RÉVÉLER LES TROUS FONCTIONNELS

Deux besoins ont été découverts en dessinant, et non en analysant.

L'écran de **validation de présence** a fait apparaître qu'une validation sans borne temporelle n'a pas de sens : si l'on peut valider à tout moment, la libération automatique devient impossible. D'où la fenêtre de dix minutes, qui n'était pas dans l'analyse initiale.

L'écran d'**arbitrage des conflits** a fait apparaître le besoin d'une référence lisible et prononçable pour chaque conflit. Un identifiant UUID ne se lit pas à voix haute dans un couloir ; `#CONF-8492` si.

FIXER LE SYSTÈME DE DESIGN

Le troisième usage est le plus classique : les maquettes ont défini les jetons de couleur, les espacements, les états et les composants récurrents. Cela a permis d'écrire les composants React une fois pour toutes, plutôt que de styler chaque page.

3.2 Inventaire des écrans

Vingt écrans ont été maquetés, répartis en trois familles. Ils ont donné, à l'implémentation, **52 écrans React** — un écran maqueté produisant parfois plusieurs vues (liste, détail, création, édition).

Famille	Écrans
Public	Page de présentation, connexion utilisateur, connexion administration, création de compte, mot de passe oublié, mentions légales
Utilisateur	Tableau de bord, catalogue des salles, fiche de salle, tunnel de réservation (4 étapes), mes réservations, détail d'une réservation, validation de présence, notifications, profil, statistiques personnelles
Administration	Tableau de bord d'occupation, parc (bâtiments, étages, plans, salles), comptes et rôles, règles de réservation, arbitrage des conflits, support et FAQ, réglages de l'assistant, journal d'audit, rapports et exports

3.3 Système de design

Le produit est en **thème sombre**. Choix assumé pour un outil consulté plusieurs fois par jour, souvent dans des salles peu éclairées, et souvent sur mobile en marchant d'un bâtiment à l'autre.

Il repose sur des **jetons** — fond, surface, contenu, accent, états — déclarés une fois et consommés par tous les composants. Un statut de salle (`disponible` , `maintenance` , `indisponible`) porte donc la même couleur partout, sans qu'aucun écran ne la redéfinisse.

Les composants récurrents ont été extraits dès le maquetage : carte, section, badge de statut, barre d'occupation, sélecteur de créneau, modale de confirmation avec champ de motif, zone de danger. Le dépôt en compte 152.

UNE RÈGLE D'INTERFACE APPLIQUÉE DEUX FOIS

Un bouton présent qui échoue à chaque clic est pire que pas de bouton — il fait croire à une panne là où il n'y a qu'une fonctionnalité non écrite.

Cette règle, posée d'abord pour le bouton de connexion Google (masqué tant que l'identifiant client n'est pas configuré), a plus tard servi à retirer les formats d'export PDF et Excel du panneau de rapports : ils y figuraient, et la couche de données les refusait systématiquement.

3.4 Le tunnel de réservation

Le tunnel comporte quatre étapes, et cet ordre a été discuté au maquetage.

#	Étape	Contenu et justification
1	Le besoin	Date, plage horaire, effectif, équipements souhaités. Aucune salle n'est encore affichée.
2	La salle	Uniquement celles qui sont libres et conformes aux règles, ordonnées par score de recommandation, avec l'explication du score.
3	Le créneau	Confirmation de l'horaire exact, battement appliqué, contraintes d'ouverture vérifiées.
4	La confirmation	Objet, participants, récapitulatif, conditions d'accès.

Montrer les salles avant d'avoir recueilli le besoin obligerait l'utilisateur à filtrer lui-même une liste dont la plupart des entrées lui sont inutiles. L'ordre retenu fait porter ce travail au système — ce qui est exactement l'objet du produit.

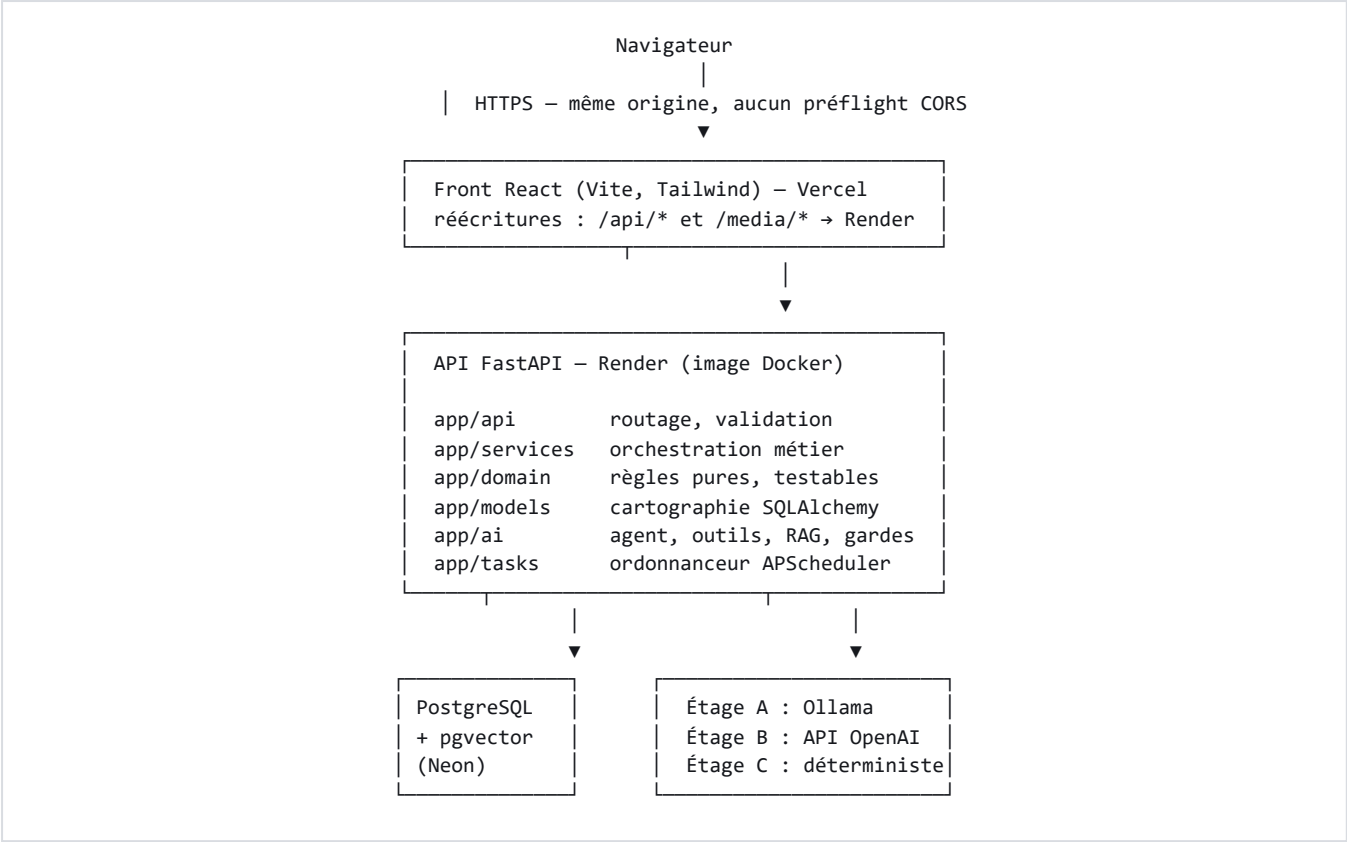
3.5 Du maquettage au code

Le passage s'est fait composant par composant plutôt qu'écran par écran. Concrètement : les jetons d'abord, puis les primitives (carte, badge, bouton, champ), puis les composants métier (sélecteur de créneau, carte de salle, frise de réservation), puis les écrans qui les assemblent.

Le bénéfice s'est mesuré tard dans le projet : lorsqu'il a fallu ajouter l'affichage du repère de localisation sur la fiche de salle côté utilisateur, la modification a tenu en quelques lignes, le composant d'affichage de plan existant déjà et étant utilisé par trois écrans d'administration.

4 Architecture technique

4.1 Vue d'ensemble



4.2 Découpage en couches

Le back-end est découpé en quatre couches, avec une règle de dépendance stricte : **chacune ne connaît que la suivante, jamais la précédente.**

Couche	Responsabilité	Interdit
app/api	Routage, validation d'entrée, sérialisation de sortie, dépendances d'authentification, en-têtes de cache	Toute règle métier
app/services	Orchestration : lire, appliquer les règles, écrire, journaliser l'audit, programmer les notifications (17 services)	Connaître HTTP
app/domain	Cœur métier pur : créneaux libres, application des règles, détection de conflits, pondération des recommandations	Importer SQLAlchemy, FastAPI, ou quoi que ce soit d'extérieur
app/models	Cartographie SQLAlchemy et rien d'autre	Contenir de la logique

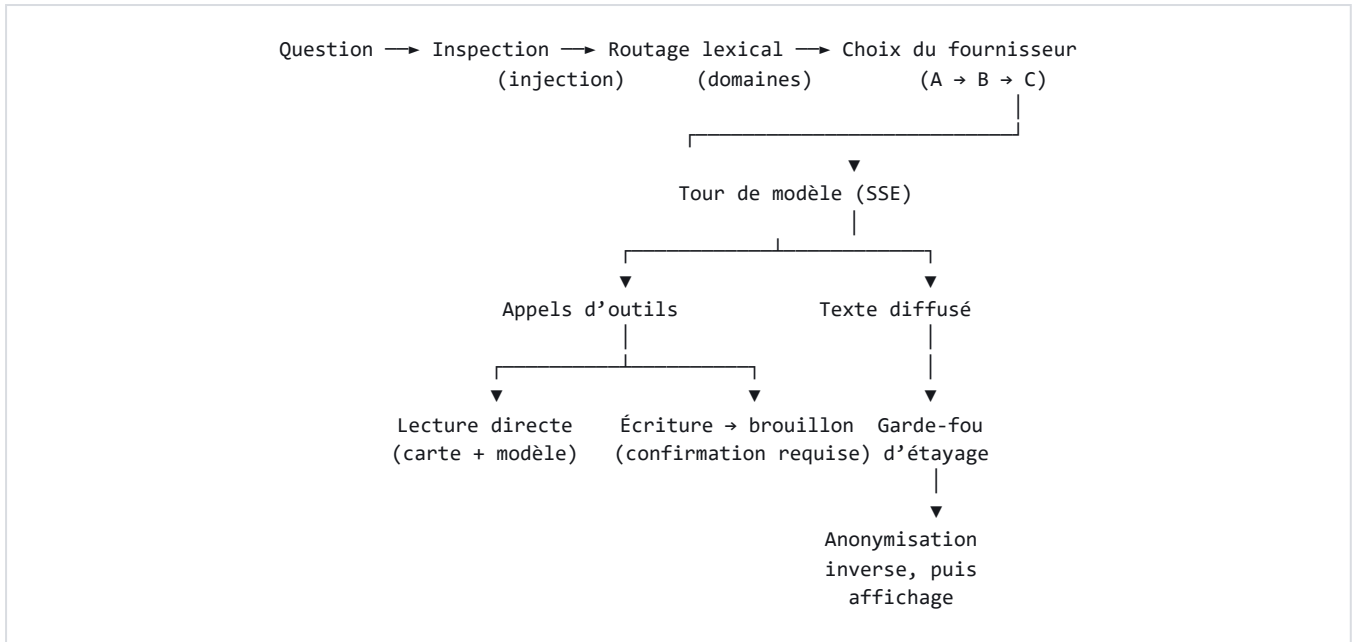
C'est la pureté de `app/domain` qui permet d'en exiger **100 % de couverture de branches** sans monter la moindre base de données : ses tests s'exécutent en millisecondes et n'ont besoin d'aucun environnement.

Le bénéfice s'est vérifié à l'usage : un défaut de pondération se corrige dans le domaine, avec un test qui tourne instantanément, sans toucher au reste de l'application.

4.3 Justification des choix

Choix	Alternative écartée	Raison
PostgreSQL	MySQL, SQLite	Seul à offrir <code>EXCLUDE USING gist</code> sur des intervalles. La garantie d'intégrité devient native au lieu d'être applicative — c'est le choix structurant du projet.
pgvector	Pinecone, Qdrant, Chroma	Une seule base à exploiter, à sauvegarder, à migrer et à déployer. Pour un projet à un développeur, la simplicité d'exploitation prime.
FastAPI	Django, Flask	Validation par annotations de type, OpenAPI généré gratuitement, et asynchrone natif — nécessaire au flux SSE de l'assistant.
SQLAlchemy 2 + psycopg3	ORM Django	Accès aux types PostgreSQL avancés : <code>tstzrange</code> , <code>vector</code> , contraintes d'exclusion.
Alembic	<code>create_all</code> au démarrage	Une migration se relit, se rejoue, se corrige et se raconte. Un <code>create_all</code> ne laisse aucune trace de l'évolution du schéma.
React + Vite	Next.js	Aucun besoin de rendu serveur ; un binaire statique se déploie sur n'importe quel hébergeur, y compris gratuit.
Tailwind	CSS modules, styled-components	Le système de design vit dans les jetons, pas dans des feuilles éparses qui divergent avec le temps.
Server-Sent Events	WebSocket	Le flux est unidirectionnel. SSE traverse les proxys sans configuration — ce qui s'est vérifié : le flux passe la réécriture Vercel sans réglage particulier.
Ollama en local	API distante uniquement	Développement hors ligne, sans coût, et sans qu'aucune donnée ne quitte le poste.

4.4 Le flux de l'assistant



Les outils d'**écriture** ne s'exécutent jamais directement. Ils produisent un brouillon conservé côté serveur sous un jeton à usage unique, que l'utilisateur doit confirmer. Au moment de l'exécution, **la sortie du modèle n'est jamais relue** : c'est le brouillon validé qui part au service métier. Un modèle qui aurait changé d'avis entre les deux tours ne peut donc rien écrire.

5 Modèle de données

46

TABLES

13

MIGRATIONS

7

DOMAINES

768

DIMENSIONS

5.1 Volumétrie et domaines

Domaine	Tables principales
Parc	buildings, floors, floor_plans, rooms, room_photos, equipments, room equipments, room_placements
Réservations	bookings, booking_participants, booking_series, booking_access_codes, access_requests, conflicts
Règles	booking_rules, opening_hours, closures
Comptes	users, user_preferences, admin_accounts, admin_permissions, permissions, admin_invitations, refresh_tokens, password_reset_tokens
Support	faq_categories, faq_articles, faq_fragments, tickets, ticket_messages, response_templates, chatbot_intents, chatbot_intent_keywords
Notifications	notifications, notification_templates
Assistant	chat_conversations, chat_messages, chat_tours
Traçabilité	audit_logs

5.2 La contrainte qui porte l'intégrité

```
ALTER TABLE bookings ADD CONSTRAINT ex_bookings_no_overlap
EXCLUDE USING gist (
    room_id WITH =,
    tstzrange(start_at, end_at, '[') WITH &&
) WHERE (status <> 'annulee');
```

Cette contrainte est le cœur de l'intégrité du produit. Elle rend **impossible** l'enregistrement de deux réservations qui se chevauchent dans la même salle. Pas improbable : impossible — quelle que soit la concurrence, quel que soit le chemin d'écriture, y compris une insertion faite à la main en SQL par un administrateur de base.

LE CHOIX DES BORNES

'[' — borne de début incluse, borne de fin exclue. Une réservation de 9 h à 10 h et une autre de 10 h à 11 h peuvent donc coexister, ce qu'attend l'intuition. Avec ']', elles se chevaucheraient à 10 h précises et la seconde serait refusée.

LA CLAUSE CONDITIONNELLE

`WHERE (status <> 'annulee')` exclut les réservations annulées. Une annulation doit libérer le créneau *sans effacer la trace* : la ligne reste, l'historique est préservé, mais elle cesse d'occuper l'espace-temps.

CE QUE CETTE CONTRAINTE A RÉELLEMENT EMPÊCHÉ

La suite de tests de concurrence lance dix demandes simultanées sur le même créneau. Sans la contrainte, plusieurs passeraient. Avec elle, exactement une réussit, et les neuf autres reçoivent une erreur applicative traduite en message métier. Le test s'exécute en 14 secondes — le temps de la sérialisation, qui fait précéder son office.

5.3 Recherche vectorielle

La table `faq_fragments` porte une colonne `vector(768)`, indexée en **HNSW** avec l'opérateur `vector_cosine_ops`.

```
CREATE INDEX ix_faq_fragments_vecteur ON faq_fragments
USING hnsw (vecteur vector_cosine_ops);
```

Le choix du **cosinus** plutôt que de la distance euclidienne a eu une conséquence heureuse au déploiement : le cosinus ignore la norme du vecteur, ce qui rend sans conséquence la troncature d'un modèle produisant 3 072 dimensions vers les 768 attendues (voir incident n° 21).

La dimension est **figée par la migration 0007**. En changer impose une migration, jamais un simple redémarrage — contrainte qui a directement dicté un paramètre du client distant.

5.4 Stratégie de migration

Treize migrations, de `0001_schema_initial` à `0013_gabarit_reactivation`. Deux règles ont été appliquées sans exception.

Une migration déjà appliquée ne se réécrit pas. Elle se corrige par la suivante. La migration `0011` existe uniquement pour renommer une contrainte que `0010` avait mal nommée ; réécrire `0010` aurait cassé toutes les bases où elle était déjà passée, production comprise.

Un gabarit fonctionnel est posé par migration, pas seulement par le jeu de démonstration. La raison est subtile et a coûté un défaut réel : la fonction de notification ignore en silence un code de gabarit absent. Une fonctionnalité livrée sans son gabarit n'aurait donc rien fait, sur une base déjà installée, sans la moindre erreur pour le signaler.

6 Réalisation — back-end

128

ROUTES

17

SERVICES

22

DOMAINES D'API

1 102

TESTS

6.1 Authentification et sessions

Mécanisme	Réglage	Justification
Jeton d'accès (JWT)	15 minutes	Court par choix : un jeton volé a une durée de nuisance bornée.
Jeton de rafraîchissement	Cookie <code>httpOnly</code> , 30 jours, chemin <code>/api/v1/auth</code>	Inaccessible au JavaScript ; le chemin restreint limite sa diffusion aux seules requêtes qui en ont besoin.
Rotation	À chaque usage	Le jeton précédent est révoqué immédiatement.
Détection de rejeu	Révocation de la famille	Rejouer un jeton déjà tourné révoque <i>toute</i> la famille : la session tombe. C'est le comportement attendu face à un vol.
Limitation	5 connexions/min, 3 réinitialisations/h	Freine le bourrage d'identifiants sans gêner l'usage normal.

DEUX PORTES DISTINCTES

`/auth/login` ouvre une session de portée *utilisateur* ; `/auth/admin/login` une session d'*administration*. Un compte administrateur connecté par la porte des utilisateurs n'obtient **aucun** privilège.

VÉRIFICATION EN PRODUCTION

Les vingt-trois routes d'administration en lecture ont été interrogées avec les deux types de session. Résultat : **403** sur vingt-deux d'entre elles avec une session utilisateur, **200** sur les vingt-trois avec une session d'administration. Le cloisonnement fonctionne, et il est démontrable.

CONNEXION GOOGLE

Par jeton d'identité vérifié contre les clés publiques de Google. Le code secret du client **n'est pas utilisé** : le navigateur obtient un jeton, le serveur le valide. Un domaine de messagerie peut être exigé — vide, tout compte Google est accepté ; `ece.fr`, `edu.ece.fr` fermerait l'application à la seule école.

6.2 Permissions

Sept permissions nommées, attribuées individuellement :

Permission	Portée
<code>rooms.manage</code>	Bâtiments, étages, plans, salles, photos, équipements
<code>rules.configure</code>	Règles de réservation, horaires, fermetures
<code>conflicts.arbitrate</code>	Arbitrage des demandes concurrentes
<code>support.handle</code>	Tickets, FAQ, gabarits de réponse, intentions de l'assistant
<code>users.manage</code>	Comptes, suspensions, retraits, invitations d'administrateurs
<code>data.export</code>	Exports CSV des statistiques et de l'audit
<code>system.configure</code>	Réglages de l'assistant, paramètres système

6.3 Concurrency et interblocages

La contrainte d'exclusion garantit l'intégrité. Elle ne suffit pas à produire une *bonne erreur*.

Dix demandes simultanées sur le même créneau concluent toutes que la salle est libre, puis s'affrontent sur la contrainte. PostgreSQL arbitre, mais au-delà de deux ou trois concurrents il détecte un cycle d'attente et sacrifie une transaction :

```
psycopg.errors.DeadlockDetected
CONTEXT: while checking exclusion constraint on tuple (0,119)
in relation "bookings"
```

Le service ne traduisait que le code `23P01` (violation d'exclusion). Le `40P01` (interblocage) remontait tel quel, et l'utilisateur recevait une erreur technique là où il devait lire que le créneau venait d'être pris.

LA CORRECTION RETENUE

```
SELECT pg_advisory_xact_lock(hashtext('salle:' || :room_id));
-- puis seulement : verification de disponibilite, puis insertion
```

Un verrou consultatif par salle, pris **avant la vérification** et non juste avant l'insertion : c'est la séquence entière *lire-puis-écrire* qui doit être indivisible. Le verrou est porté par la transaction, donc relâché au `COMMIT` comme au `ROLLBACK`. Avec un verrou unique par salle, aucun cycle d'attente n'est possible.

POURQUOI PAR SALLE ET NON PAR CRÉNEAU

La contrainte parle de **chevauchement**, pas d'égalité. Une clé de verrou par créneau laisserait passer de front deux réservations qui se recouvrent partiellement — 9 h–11 h et 10 h–12 h ont des clés différentes mais se chevauchent. Le verrou doit donc porter sur la seule chose qu'elles ont en commun : la salle.

6.4 Ordonnanceur

Tâche	Période	Rôle
Libération, clôture et rappels	5 min	Libère les créneaux sans présence validée, clôt les réservations passées, envoie les rappels et vide la file de courriels
Rafraîchissement des agrégats	15 min	Recalcule la vue matérialisée d'occupation
Purge des jetons expirés	quotidienne	Supprime les jetons de session et de réinitialisation périmés

6.5 Moteur de recommandation

Première brique Data du projet. Il ordonne les salles compatibles selon un score composite **explicable** : chaque composante est calculée séparément et exposée, de sorte que l'interface peut justifier le classement.

Composante	Ce qu'elle capture
Adéquation de capacité	Une salle de 90 places pour 4 personnes est un gâchis ; une salle de 4 places pour 4 personnes est juste. La composante pénalise l'écart dans les deux sens.
Équipements	Présence des équipements explicitement demandés.
Proximité	Bâtiment habituel de la personne, renseigné à l'inscription — d'où l'écran d'accueil qui le demande.
Accessibilité	Conformité PMR quand elle est requise.
Occupation	Taux d'occupation de la salle, pour répartir l'usage plutôt que saturer toujours les mêmes.

Le calcul vit dans `app/domain/recommendation.py`, sans aucune dépendance externe, et est couvert à 100 % de ses branches.

6.6 Notifications et courriels

Deux canaux, un seul déclencheur. Chaque événement métier — confirmation, rappel, annulation, code d'accès, suspension, réactivation — produit une notification applicative **et** un courriel, tous deux rendus depuis un gabarit stocké en base et modifiable depuis l'administration, sans redéploiement.

UN DÉFAUT QUI N'ÉMETTAIT AUCUNE ERREUR

L'expédition est programmée en **tâche de fond de la requête** (`BackgroundTasks`), et non laissée à l'ordonnanceur.

Sans cela, le message est déposé dans la file du processus et personne ne la vide : il ne part qu'au passage suivant de l'ordonnanceur — jusqu'à cinq minutes plus tard — et jamais du tout si celui-ci est arrêté. Un administrateur qui vérifiait aussitôt ne voyait rien arriver, *sans la moindre erreur au journal*, le relais n'ayant rien reçu.

Le courriel ne suit qu'un **changement réel d'état** : réappliquer un statut déjà porté n'apprend rien à personne, et un courriel de trop est un courriel de moins qu'on lit.

7 Réalisation — front-end

52

ÉCRANS

152

COMPOSANTS

474

TESTS

90,25 %

COUVERTURE

7.1 Organisation

```
src/
  api/          couche d'accès HTTP, un module par domaine
  components/   composants réutilisables, par famille
  pages/        écrans, par espace (public, user, admin)
  hooks/        état partagé, requêtes, formulaires
  utils/        dates, formats, calculs d'affichage
  router.jsx    routes et gardes
```

7.2 Couche d'accès et adaptateurs

La couche `api/` est la **seule** à connaître l'existence du réseau. Aucun composant n'appelle `fetch` directement.

Elle porte aussi les **adaptateurs**, qui traduisent les charges utiles de l'API — nommées en `snake_case` — vers les objets attendus par l'interface. Cette frontière a permis, à plusieurs reprises, d'ajouter un champ côté serveur sans toucher au moindre écran : c'est ainsi que `floor_has_plan` a été introduit pour supprimer une requête vouée à l'échec.

7.3 Gardes de routage

Trois familles de routes : publiques, utilisateur authentifié, administration authentifiée. La racine `/` redirige selon l'état de session — tableau de bord si une session est ouverte, page de présentation sinon.

L'administration a sa propre page de connexion, `/admin/connexion`, cohérente avec la séparation des portes côté serveur décrite en 6.1.

7.4 Un piège de fuseau horaire

La répartition des réservations par tranche horaire lisait `Date.getHours()` — c'est-à-dire l'heure **du poste** — alors que les tranches 08–10, 10–12, 14–16 et 16–18 sont celles du campus.

La chaîne d'intégration continue tourne en UTC. Une réservation de 9 h y devenait 7 h, ne tombait dans aucune tranche, et la page annonçait qu'il n'y avait pas assez de réservations sur un jeu qui en contenait 608.

CE N'ÉTAIT PAS UN TEST FRAGILE

Un navigateur réglé sur un autre fuseau — un utilisateur en déplacement, un poste mal configuré — affichait exactement la même page vide. La correction lit l'heure dans le fuseau de l'établissement via `Intl.DateTimeFormat`, sans dépendance nouvelle.

```
const FUSEAU_CAMPUS = 'Europe/Paris';

export function heureCampus(date) {
  return Number(
    new Intl.DateTimeFormat('en-GB', {
      timeZone: FUSEAU_CAMPUS, hour: '2-digit', hourCycle: 'h23',
    }).format(date)
  );
}
```

8 L'assistant conversationnel

8.1 Catalogue d'outils

L'assistant répond en s'appuyant sur des **outils** qui interrogent la vraie base — jamais sur ce qu'un modèle croit savoir. Quatorze outils sont enregistrés :

Domaine	Outils	Nature
Parc	rechercher_salles , recommander_salle , localiser_salle , consulter_disponibilite	lecture
Réservations	lister_mes_reservations , obtenir_code_acces , consulter_regles	lecture
Réservations	creer_reservation , modifier_reservation , annuler_reservation	écriture
Assistance	rechercher_faq , transferer_humain	lecture
Assistance	creer_ticket	écriture

8.2 Architecture à trois étages

Étage	Où	Rôle
A — Ollama	Poste de développement	Gratuit, hors ligne, aucune donnée ne sort. qwen2.5:7b et nomic-embed-text .
B — API compatible OpenAI	Production	N'existe que parce qu'aucun hébergement gratuit ne fait tourner un modèle de sept milliards de paramètres.
C — Moteur déterministe	Toujours disponible	Ce n'est pas une panne : un moteur de repli qui reconnaît des intentions déclarées en base, appelle les mêmes outils, et répond sans aucun modèle.

Le passage d'un étage à l'autre est journalisé avec son **déclencheur**, ce qui permet au tableau de bord d'administration de distinguer un repli subi d'un repli choisi.

8.3 Un repli parfois choisi

Une mesure a modifié la conception. Sur cinq essais identiques de « donne-moi mes prochaines réservations », pour un compte qui en possède cinq :

Moteur	Appelle l'outil	Réponse
qwen2.5:7b	3 fois sur 5	variable
qwen2.5:14b	0 fois sur 3	fausse
Moteur déterministe	2 fois sur 2	juste

Quand le modèle n'appelle rien, il écrit une phrase plausible — parfois « je n'ai pas trouvé » — devant un agenda plein. **L'utilisateur ne peut pas distinguer cette réponse d'une vraie.**

UN CORRECTIF ESSAYÉ PUIS ABANDONNÉ

Première tentative : exécuter la lecture avant que le modèle ne parle, et lui remettre le résultat. L'outil partait bien 5 fois sur 5, mais deux essais répondaient encore « je n'ai pas trouvé cette information » *à côté de la carte qui listait les réservations*. Le modèle contredisait la donnée qu'il avait sous les yeux.

CONCLUSION RETENUE

Le défaut n'est pas le modèle, c'est de lui laisser ce choix. **Une question fermée dont la réponse est une liste n'a rien à gagner d'un modèle de langage et tout à perdre.** Elle part désormais au moteur déterministe, et le journal la distingue par son déclencheur `intention_certaine` — pour qu'elle ne soit pas comptée comme une panne dans les statistiques.

Le déclenchement reste étroit : le pluriel possessif est exigé et les verbes d'écriture excluent, pour qu'« annuler ma réservation » garde son tour de confirmation. Douze cas de bord le verrouillent, plus une contre-épreuve vérifiant qu'une question ouverte consomme bien un tour de modèle.

8.4 Recherche documentaire hybride

La base de connaissances est découpée en fragments, vectorisés et stockés en base. La recherche additionne deux volets, parce qu'ils échouent différemment :

- le **vectorel** retrouve « je n'arrive pas à entrer dans la salle » sous l'article « Code d'accès », sans partager un seul mot ; il rate en revanche un identifiant, un nom propre ou un mot rare, que l'embedding dilue ;
- le **lexical** trouve le mot exact et rien d'autre ; il ne sait pas qu'annuler et supprimer sont proches.

La fusion se fait par **rangs réciproques** (RRF) plutôt que par somme de scores. Une similarité cosinus et un `ts_rank` ne vivent pas sur la même échelle : les additionner reviendrait à comparer des degrés et des kilomètres. Les rangs, eux, sont comparables par construction.

MESURE DE L'APPORT, SUR LE CORPUS RÉEL

Question	Lexical seul	Avec vecteurs
« comment je prévois que je suis arrivé »	aucun résultat	Valider ma présence sur place
« ma réunion est annulée que faire »	À quoi sert SmartRoom	Modifier l'horaire ou la salle
« je n'arrive pas à entrer dans la salle »	À quoi sert SmartRoom	Valider ma présence sur place

La première question ne partage **aucun mot** avec l'article qui y répond.

Sans modèle de vecteurs joignable, la fonction de vectorisation rend `None` et la recherche retombe sur son seul volet lexical. **C'est un mode dégradé prévu, pas un échec** — et c'est exactement ce qui s'est produit en production avant que l'étage B ne soit configuré.

8.5 Garde-fous

Garde-fou	Ce qu'il empêche
Injection	Les délimiteurs sont neutralisés, les motifs suspects comptés dans le journal du tour. Une consigne cachée dans une question ne devient pas une instruction.
Étayage	La réponse est confrontée aux données réellement obtenues. Si elle avance ce qui n'y figure pas, une réserve est ajoutée. C'est ce garde-fou qui fait dire « je n'ai pas trouvé cette information » plutôt que d'inventer un chiffre.
Anonymisation	Avant tout envoi à un fournisseur distant, adresses, noms et téléphones sont remplacés par des jetons stables le temps de la conversation. La table de correspondance ne quitte jamais le serveur.
Adresses de fichiers	Toute clé se terminant par <code>_url</code> est retirée de ce que le modèle relit, et n'existe que dans la carte que le front sait rendre.

DEUX DÉCISIONS DE CONCEPTION DANS CES GARDE-FOUS

Le fournisseur distant refuse d'émettre tant qu'aucune fonction d'anonymisation n'est branchée. Un oubli de configuration ne doit pas se traduire par l'envoi de noms à un tiers. Cette garde était juste — et son absence de câblage a rendu tout un étage inatteignable (incident n° 19).

La règle sur les adresses vit dans le type de résultat, et non chez chaque outil. Un outil peut oublier de signaler qu'il rend une adresse ; ce fichier-là ne peut pas oublier de la retirer.

8.6 Budget et observabilité

Chaque tour est borné : budget de contexte (9 200 jetons), nombre d'itérations (5), nombre d'outils par tour (8), durée totale (25 s), délai de premier jeton (6 s).

Le journal du tour enregistre : le mode, le modèle, le nombre d'itérations et de relances, les outils appelés et leur durée, le délai de premier jeton, les injections suspectées, les délimiteurs neutralisés, et la troncature éventuelle. Un écran d'administration l'expose.

```
{ "mode": "modele", "modele": "gemini-3.5-flash-lite", "repli": false,  
  "iterations": 2, "outils": ["localiser_salle"], "duree_ms": 1559,  
  "premier_jeton_ms": 1124, "injection_suspectee": false, "tronque": false }
```

9 Sécurité et conformité

9.1 Gestion des secrets

Aucun secret n'est versionné. Le fichier `.env` est ignoré par git, et le modèle `.env.example` documente chaque variable sans en porter la valeur.

Un fichier `.env` avait été committé aux premiers temps du projet. Il a été **retiré de tout l'historique** :

```
git filter-branch -f --index-filter \  
    "git rm --cached --ignore-unmatch smartroom-api/.env" --prune-empty -- --all  
git reflog expire --expire=now --all  
git gc --prune=now  
git push -f origin main
```

CE QUE LA RÉÉCRITURE NE FAIT PAS

Elle retire la trace, elle **n'annule pas l'exposition**. Tous les identifiants concernés ont donc été régénérés : mot de passe de base de données, mot de passe d'application de messagerie, clés d'API. C'est la régénération qui protège, pas la réécriture.

9.2 Cloisonnement des données personnelles

Un défaut réel a été trouvé et corrigé, et il mérite d'être détaillé car il illustre une erreur de raisonnement courante.

La route `/stats/me` portait `Cache-Control: private, max-age=300`. La directive semblait suffisante : `private` interdit aux intermédiaires de conserver la réponse ; seul le navigateur du destinataire la garde.

Mais un cache navigateur est indexé par URL, et `/stats/me` est la même URL pour tout le monde. Deux comptes ouverts successivement sur le même poste, à moins de cinq minutes d'intervalle, et le second lisait les chiffres du premier : nombre de réservations, heures réservées, annulations, salles fréquentées.

CONSTAT ET VÉRIFICATION

Observé sur deux comptes affichant les mêmes dix réservations et les mêmes huit annulations. Vérifié en base : l'un des deux n'en avait aucune, et la réservation affichée appartenait au seul autre compte, dont il n'était même pas participant.

`Vary: Authorization` isolerait les réponses, mais le jeton tourne à chaque rafraîchissement : le cache serait manqué à tous les coups. Autant ne rien garder, et le dire. La route est passée à `private, no-store`, ainsi que l'export CSV personnel — qui portait les mêmes données ligne à ligne, *sans aucun en-tête*.

Les chiffres publics gardent leur cache : anonymes, identiques pour tous, rien n'y désigne personne. Un test le vérifie explicitement, sans quoi on supprimerait les deux d'un même geste et la page d'accueil rechargerait ses agrégats à chaque visite, pour rien.

9.3 Retrait d'un compte par anonymisation

La suspension empêche de réserver et se défait. Il manquait le geste définitif.

Effacer la ligne casserait le journal d'audit, les frises de réservation et les agrégats d'occupation, qui référencent tous ce compte. **Ce que le règlement demande n'est pas la disparition de l'historique : c'est celle de l'identité.**

Ce qui est effacé	Ce qui est conservé
Adresse, nom, prénom, téléphone, promotion, service, badge, photo de profil, mot de passe	Réservations passées (sans nom), agrégats d'occupation, entrées d'audit, séries

L'adresse devient `compte-retire-<12 caractères>@anonyme.invalid`. Le domaine `.invalid` est réservé par la **RFC 2606** : aucune remise n'est possible, donc aucun envoi accidentel n'atteindra une boîte réelle.

Les réservations à venir sont annulées et les créneaux libérés — les laisser occuperait des salles au nom de quelqu'un qui n'existe plus. Les sessions ouvertes sont révoquées.

Trois refus précèdent toute écriture, chacun avec son test : un motif vide, un compte porteur de droits d'administration (les retirer est une décision distincte, avec son écran et sa trace), et le compte de l'auteur de la demande.

9.4 Journal d'audit

Toute décision d'administration — création, modification, suppression, suspension, réactivation, arbitrage, dépôt de document — est enregistrée avec son auteur, sa cible, son action, son motif et son horodatage. L'écran d'audit permet de la relire, de la filtrer et de l'exporter en CSV.

10 Stratégie de tests

10.1 Cinq niveaux, cinq intentions

Niveau	Ce qu'il éprouve	Dépendances
Domaine	Les règles métier, en isolation totale	Aucune
Services et API	L'intégration réelle : contrainte d'exclusion, JWT, transactions	PostgreSQL
Assistant	La boucle d'agent, les outils, les garde-fous, les bascules	Base + fournisseur simulé
Front	Les écrans et leur logique	MSW intercepte le réseau
Bout en bout	Les parcours complets dans un vrai navigateur	Pile complète

10.2 Chiffres

1 576 TESTS AU TOTAL	1 102 BACK-END	474 FRONT-END	100 % DOMAINE
-------------------------	-------------------	------------------	------------------

Suite	Tests	Couverture	Fichiers
Back-end complet	1 102	—	46
· dont Domaine	—	100 % des branches (seuil imposé)	—
· dont Services et API	545	87,59 %	—
· dont Assistant	159	84,71 %	—
Front-end	474	90,25 % (mesurée en UTC)	46
Bout en bout	3 parcours	—	3

Relevés le jour de la rédaction, sur la pile complète. Tous verts.

10.3 Le principe de la contre-épreuve

Un test qui ne peut pas échouer ne prouve rien. Chaque correctif d'importance est accompagné d'une vérification que le test échoue bien sans lui. Plusieurs messages de commit en portent la trace explicite :

- « Sans le correctif, la trace porte deux cartes au lieu d'une. »
- « Vérifiés en retirant l'envoi, trois d'entre eux échouent. »
- « Contre-épreuve sans le relevé : zéro plan. »

- « Sans le correctif : `KeyError: 'extra_content'` . »

UNE CONTRE-ÉPREUVE A INVALIDÉ UN CORRECTIF

Un test censé prouver une correction passait *aussi sans elle*. Le motif de remplacement n'avait rien changé : le formateur automatique avait replié l'expression sur trois lignes, et la chaîne recherchée n'existait plus telle quelle. Sans contre-épreuve, le correctif aurait été committé comme vérifié alors qu'il n'était pas appliqué.

10.4 Ce que les tests ont réellement attrapé

Un module du domaine à 0 %. `app/domain/organisation.py` n'était éprouvé que depuis les tests de services, où le travail « Domaine » ne regarde pas. Le module apparaissait à 0 % et le seuil de 100 % échouait. La leçon : *un module du domaine se prouve dans le domaine*.

Une exclusion inopérante. Un test d'anonymisation a montré qu'une exclusion écrite en rétrospection ne servait à rien : la correspondance commençait sur le mot exclu, et ce qui la précédait était un article.

Une contradiction entre deux tests. Le parcours de bout en bout cherchait un bouton « Ouvrir la navigation » quand le test unitaire cherchait « Menu », et tous deux passaient sur leur propre périmètre. Le déclencheur portait le mot « Menu » écrit, et non seulement lu par les lecteurs d'écran — trois traits ne disent pas qu'ils cachent dix-huit écrans. Lui donner un `aria-label` différent aurait masqué ce texte visible, contre WCAG 2.5.3 : c'est donc le test de bout en bout qui a suivi l'implémentation.

11 Intégration continue

11.1 Composition de la chaîne

GitHub Actions, cinq travaux : **Domaine, Services et API, Assistant, Front, Bout en bout**. Chacun impose son seuil de couverture. La chaîne exécute également `ruff check`, `ruff format --check` et `alembic check` — ce dernier vérifiant que le schéma décrit par les modèles correspond aux migrations.

11.2 Trois causes d'échec indépendantes

La chaîne a échoué durablement, et les causes n'avaient aucun rapport entre elles.

L'IMAGE POSTGRESQL N'AVAIT PAS PGVECTOR

La migration `0007` crée l'extension `vector`, absente de `postgres:16-alpine`. `alembic upgrade head` s'arrêtait là, *avant même d'atteindre les tests* : le travail « Services et API » échouait depuis huit exécutions, sans rapport avec les commits qui les déclenchaient.

Mesuré, et non supposé : `docker run --rm postgres:16-alpine` ne contient aucun fichier d'extension `vector`. Les cinq autres extensions du schéma vivent dans `contrib` et sont partout ; celle-ci non.

UNE CONVENTION DE NOMMAGE DOUBLEMENT APPLIQUÉE

La migration `0010` passait le nom complet à `create_check_constraint`, que la convention de nommage préfixait une seconde fois :

```
base      : ck_booking_rules_ck_booking_rules_notice_non_vide
modele    : ck_booking_rules_notice_non_vide
```

`alembic check` s'arrêtait sur cet écart. Une migration `0011` corrective a été écrite plutôt que de réécrire `0010`.

UN ÉPINGLAGE DE DÉPENDANCE MENSONGER

`requirements.txt` épinglait `fastapi==0.118.0` alors que le code était développé et testé contre `0.141.1`. Sous la version épinglée, FastAPI ne résout plus certaines annotations et prend le modèle Pydantic et la dépendance de session pour des champs de corps :

```
422 « payload est obligatoire, session est obligatoire »
```


LE DÉFAUT LE PLUS DANGEREUX DU PROJET

Tout passait en local, où l'environnement virtuel portait `0.141.1`, et nulle part ailleurs. Le `Dockerfile` installant depuis `requirements.txt`, **la production aurait servi une API cassée sur ces routes** — et le développement n'aurait rien montré.

Établi en reconstruisant l'environnement de la chaîne : Linux, Python 3.12, `TZ=UTC`, base en collation C. L'échec s'y reproduit et disparaît avec la bonne version.

11.3 Deux pièges d'environnement

LE BIT EXÉCUTABLE

`docker-entrypoint.sh` était stocké en `100644`. Le `Dockerfile` lui applique bien un `chmod +x`, mais le service monte le répertoire source sur `/app` : le montage recouvre le répertoire de l'image, et sous Linux le fichier reprend son mode git.

```
exec: "/app/docker-entrypoint.sh": permission denied
```

Invisible sous Windows, où Docker Desktop présente les fichiers partagés comme exécutables. Et `core.fileMode` valant `false` sur ce dépôt, git n'avait jamais enregistré le bit posé sur le disque — d'où `git update-index --chmod=+x`, qui l'inscrit explicitement.

LE LIMITEUR DE CONNEXION

`.env.example` porte la valeur de production, cinq par minute, et la chaîne le copie en `.env` : cela écrasait le plafond relevé de `docker-compose.yml`. La campagne de bout en bout ouvre une session par test — elle ne peut pas en partager une, car le jeton tourne à chaque chargement de page et rejouer un cookie déjà tourné déclenche la détection de rejeu — et tombait donc systématiquement sur le limiteur.

L'étape pose désormais la variable dans son environnement, qui l'emporte sur le fichier sans le modifier.

12 Fonctionnement en local

12.1 Démarrage de la pile

```
# 1. Services d'infrastructure
docker compose up -d db courriel

# 2. Schema et jeu de demonstration
cd smartroom-api
..\venv\Scripts\python.exe -m alembic upgrade head
..\venv\Scripts\python.exe -m scripts.seed

# 3. API
..\venv\Scripts\python.exe -m uvicorn app.main:app --reload --port 8000

# 4. Front (autre terminal)
cd smartroom-front
npm run dev           # http://localhost:5180
```

La boîte de développement intercepte tous les courriels : rien ne quitte la machine, quelle que soit l'adresse écrite sur le message. Elle est consultable sur `http://localhost:8025`.

12.2 Le préchauffage des modèles

Ollama ne garde les poids en mémoire que le temps défini par `keep_alive` — trente minutes ici — puis les recharge depuis le disque.

MESURE SUR LE POSTE DE DÉVELOPPEMENT	
Appel	Durée
Premier appel, poids sur disque	79 secondes
Appels suivants, poids en mémoire	1,1 seconde

Le budget de premier jeton étant de six secondes, **toute première question d'une session partait au repli déterministe**, qui ne connaît qu'une poignée d'intentions : l'assistant répondait « je n'ai pas compris » à des questions que le modèle traite sans peine.

La méthode de préchauffage existait et se disait « appelée au démarrage de l'application » ; *rien ne l'appelait*. Elle l'est désormais, sur deux modèles seulement — celui du raisonnement et celui des vecteurs. Précharger trois modèles saturerait la mémoire d'un poste ordinaire pour un gain que la mesure ne montre pas.

12.3 Le budget de premier jeton

Fixé à **six secondes**, et non à deux et demie comme au départ.

Situation	Premier jeton
Premier appel	3 455 ms
Appels suivants (invite mise en cache)	1 524 ms
Moteur déterministe	150 à 600 ms

À 2 500 ms, chaque première question d'une session partait au repli et le modèle ne servait jamais. Attendre six secondes avant d'y renoncer reste supportable.

12.4 Jeu de démonstration

Le script de peuplement crée :

6 BÂTIMENTS	15 SALLES	608 RÉSERVATIONS	24 FRAGMENTS
-----------------------	---------------------	----------------------------	------------------------

Plus cinq utilisateurs, cinq comptes d'administration aux périmètres distincts, une fermeture globale, et deux conflits en attente d'arbitrage (#CONF-8492 , #CONF-8493).

UN SEED QUI PRÉSERVE CE QU'ON LUI A CONFIE

Le script est **idempotent sur les documents déposés** : photos de bâtiment, repères de salle et plans d'étage sont relevés avant la purge et reposés après. *Un jeu de démonstration a le droit de refaire ses données, pas celui d'effacer ce qu'on lui a confié.*

Cette précaution avait été écrite pour les photos, puis oubliée lorsque les plans d'étage sont arrivés. Ils vivent dans `floor_plans`, que la purge emportait en cascade avec les bâtiments : les fichiers survivaient sur le disque, mais la ligne qui disait à quel étage ils appartenaient disparaissait, et la fiche affichait « Aucun plan déposé pour cet étage » sur un étage qui en avait un.

COMMENT LE DÉFAUT A ÉTÉ ÉTABLI

Trois fichiers présents dans `media/plans` ; des placements de salles journalisés à l'audit les 29 et 31 août — *or on ne place une salle que sur un plan existant* ; et zéro ligne dans `floor_plans`. Les trois observations ensemble ne laissent qu'une explication.

La clé de relevé est le **code** du bâtiment et de l'étage, non leur identifiant : les codes portent une contrainte d'unicité et survivent d'un jeu à l'autre, contrairement aux identifiants, régénérés à chaque purge.

13 Déploiement en production

13.1 Cible retenue

Composant	Service	Rôle et palier
Base de données	Neon	PostgreSQL 18 managé avec pgvector, palier gratuit
API	Render	Image Docker construite depuis le dépôt, palier gratuit
Front	Vercel	Statique + réécritures, palier gratuit
Supervision	UptimeRobot	Sonde toutes les 5 minutes, palier gratuit
Modèle et vecteurs	API Gemini	Façade compatible OpenAI, palier gratuit

13.2 Le front devant l'API

```
{
  "rewrites": [
    { "source": "/api/:path*",    "destination": "https://smartroom-api-ryya.onrender.com/api/:path*" },
    { "source": "/media/:path*",  "destination": "https://smartroom-api-ryya.onrender.com/media/:path*" },
    { "source": "/(.*)",          "destination": "/index.html" }
  ]
}
```

Les deux premières réécritures font que **le navigateur appelle l'API sur l'origine qui lui a servi la page**. La requête part en même origine : aucun préflight CORS n'est nécessaire, et le cookie de rafraîchissement — `SameSite=Lax` — reste attaché, ce qu'un appel direct vers `onrender.com` lui interdirait.

La troisième est indispensable au routage côté client : sans elle, un rechargement sur `/salles` ou `/admin/comptes` rendrait un 404.

UNE DISTINCTION OPÉRATIONNELLE À CONNAÎTRE

Vercel donne à **chaque déploiement** une adresse propre (`smartroommanager-<hachage>-<équipe>.vercel.app`) en plus de l'alias stable. Seul l'alias est déclaré côté Google et dans `CORS_ORIGINS` — c'est donc lui, et lui seul, qu'il faut utiliser. Une connexion Google tentée depuis l'adresse de déploiement rend `Erreur 400 : origin_mismatch`.

13.3 Point d'entrée du conteneur

L'image applique les migrations au démarrage. `alembic upgrade head` est idempotent : sur une base déjà migrée, il ne fait rien et rend 0. Le lancer à chaque démarrage évite l'étape manuelle qu'on oublie.

Deux réserves sont assumées et documentées dans le script : deux conteneurs démarrant ensemble migrent en même temps — Alembic pose un verrou sur sa table de version, le second attend puis ne trouve rien à faire — et une migration destructive appliquée automatiquement ne se relit pas. Pour un projet à un déploiement, le compromis penche du bon côté.

13.4 La sonde de démarrage

Le premier déploiement s'est arrêté sur « Base injoignable après 30 tentatives », sans rien dire de plus. Ce message laissait le choix entre **un mot de passe faux**, **un hôte inconnu**, **un pare-feu** et **une base endormie** : quatre pistes, aucune preuve. Le `2>/dev/null` de la boucle jetait le message de PostgreSQL, le seul qui tranche.

La sonde a été améliorée **quatre fois**, et chaque amélioration a permis d'avancer d'un pas.

#	Amélioration	Ce qu'elle a débloqué
1	Écrire la raison du refus (1 ^{er} , 10 ^e et dernier essai)	Le message de PostgreSQL devient lisible. Guillemets et retours à la ligne neutralisés, le journal restant du JSON.
2	Deux sorties d'échec : 1 réessayable, 2 fatale	La sonde avait répété trente fois « Base pas encore prête » pour une variable <code>CORS_ORIGINS</code> mal écrite — soixante secondes perdues, et un message désignant le mauvais coupable.
3	Troncature portée de 300 à 1 200 caractères	Le journal s'arrêtait exactement avant la liste des adresses essayées — la seule information utile, puisqu'elle dit si la bascule IPv6 → IPv4 a eu lieu.
4	Joindre les variables de connexion	Pydantic élide le milieu des chaînes longues, et l'anomalie se cache justement dans ce qu'il coupe.

UNE RÈGLE APPLIQUÉE SANS EXCEPTION

Le mot de passe ne figure jamais dans un journal. **Seule sa longueur apparaît** — `POSTGRES_PASSWORD=(16 caracteres)` — ce qui distingue un champ vide d'un champ rempli, et un champ rempli d'un champ contenant toute une chaîne de connexion, sans rien révéler.

Une correction de classement a accompagné la quatrième : `database_url` est une propriété *calculée*, et l'erreur ne surgissait qu'à sa lecture, après le contrôle de configuration. Le journal annonçait « base pas encore prête » pour une adresse mal formée qui ne guérira jamais en attendant.

13.5 Chaîne d'incidents de connexion

Chaque étape a été résolue par la mesure, jamais par la supposition.

Symptôme	Cause réelle
<code>error parsing value for field 'cors_origins'</code>	Le champ est une liste ; pydantic-settings attend du JSON , pas une URL nue.
<code>Network is unreachable (2a05:...)</code>	Le palier gratuit de Render ne sort qu'en IPv4, et Neon publie les deux familles. La bascule avait bien lieu — c'est la troncature élargie qui l'a montré — et les trois adresses IPv4 répondaient.
<code>password authentication failed</code>	Mode TLS absent. Voir ci-dessous.
<code>invalid port number</code>	Un <code>POSTGRES_HOST</code> portant déjà son port produisait <code>...neon.tech:5432:5432/neondb</code> .

LE MODE TLS, OU COMMENT UN MESSAGE DÉSIGNE LE MAUVAIS COUPABLE

```
ERROR: password authentication failed for user 'neondb_owner'
ERROR: connection is insecure (try using `sslmode=require`)
```

`libpq` applique `prefer` par défaut : il tente le chiffrement, puis *se rabat en clair* si la négociation échoue. Neon refuse alors la connexion — et sa première ligne est un message générique qui parle de mot de passe.

On cherchait donc un identifiant faux là où il manquait un mode de connexion. Le mode est désormais **déduit de l'environnement** — `require` partout sauf en local, où la base tourne dans un conteneur voisin sans certificat et où l'exiger empêcherait tout démarrage. Une variable permet d'imposer `verify-full` là où l'autorité est connue, ou `disable` sur un réseau privé déjà chiffré.

13.6 Persistance des médias

Le disque de Render est **éphémère**. Un fichier déposé depuis l'administration en ligne y survit quelques heures, puis disparaît au redéploiement suivant — alors que sa ligne en base, elle, persiste. L'écran affiche alors une image cassée, *sans rien pour le signaler*.

LE JOURNAL L'A MONTRÉ SANS AMBIGUÏTÉ

```
19:15:49 GET /media/batiments/eif1.svg      200  ← avant le depot
19:20:20 GET /media/batiments/5378...webp  404  ← apres
```

Solution retenue : les fichiers sont *versionnés*. Ils entrent dans l'image Docker et suivent le déploiement. La procédure consiste donc à déposer **en local**, l'**application locale étant branchée sur la base de production**, puis à commiter les fichiers apparus : la ligne part dans Neon, le fichier dans le dépôt, et le commit les réunit.

Le dépôt porte aujourd'hui les photos des quinze salles, les images des six bâtiments, les repères de localisation et les plans d'étage — 154 fichiers de médias au total.

Alternative	Pourquoi écartée
Disque persistant Render	Non disponible sur le palier gratuit ; exige une instance payante.
Stockage objet externe	La bonne réponse pour une application réelle. Impose de réécrire la couche de stockage — décision assumée, pas un oubli.

13.7 L'assistant en ligne

C'est la partie qui a demandé le plus de mise au point. Elle a révélé **six défauts** que le développement local ne pouvait pas montrer, puisqu'Ollama y sert seul et qu'aucun d'eux ne vit sur son chemin.

L'ÉTAGE DISTANT ÉTAIT CONFIGURABLE ET INATTEIGNABLE

Le client refuse d'émettre sans anonymisation — une garde voulue — mais *aucun* des cinq points de construction du sélecteur ne lui en passait une. On pouvait donc le configurer entièrement sans qu'il serve jamais, le seul signe étant une ligne de journal : « Fournisseur distant désactivé : anonymisation absente ».

Le défaut sûr est de masquer, pas de se taire. L'anonymiseur est branché par défaut.

LA COHÉRENCE DES VECTEURS

Poste local et hébergement partagent la même base. Les fragments portent les coordonnées que leur a données un modèle donné ; si l'un vectorise avec Ollama et l'autre avec l'étage distant, la similarité cosinus reste calculable et *compare deux espaces sans rapport*. La recherche cesse de trouver, sans erreur pour l'expliquer.

Un réglage épingle désormais ce rôle des deux côtés, et un script d'administration réindexe le corpus à chaque changement de modèle.

LA DIMENSION DES VECTEURS

MESURE SUR GEMINI-EMBEDDING-001	
Requête	Dimensions rendues
Par défaut	3 072
Avec dimensions: 768	768

La colonne pgvector étant figée à 768, sans ce paramètre l'insertion échoue au lieu de dégrader. La troncature est ici sans conséquence : la recherche compare par cosinus, une mesure qui ignore la norme.

LES APPELS D'OUTILS SIMULTANÉS FUSIONNAIENT

L'index corréle les morceaux d'un même appel d'un fragment à l'autre. La façade Gemini rend l'appel entier d'un coup et **omet cet index** : deux appels simultanés tombaient sous la même clé, noms et arguments concaténés, et `{"etat": "a_venir"}` ne se relit pas — l'assembleur écartait l'entrée. L'agent ne recevait *aucun* outil là où il en avait demandé deux. Ce n'est pas un cas de bord : le budget autorise huit outils par tour.

LES ARGUMENTS REPARTAIENT EN OBJET

Value is not a string: {"question":"absence"}

La forme utile à la boucle d’agent est un dictionnaire, et Ollama l’accepte. Le protocole OpenAI transporte une chaîne JSON. Mesure directe : la même requête rend 200 avec une chaîne et 400 avec un objet.

Le défaut ne se voyait qu’au second aller-retour, celui qui renvoie le résultat de l’outil au modèle. Le premier appel réussissait, l’outil s’exécutait, la carte s’affichait — puis le tour basculait au repli, et l’écran montrait les bons articles surmontés d’un « je n’ai pas compris la demande ».

LA SIGNATURE DE RAISONNEMENT

400 Function call is missing a thought_signature in functionCall parts.
This is required for tools to work correctly.

Les modèles Gemini 3 joignent à chaque appel d’outil une signature et exigent de la retrouver au tour suivant. Elle voyage dans extra_content , hors du protocole OpenAI, et se perdait à la traduction. Le client la retient désormais par identifiant d’appel et la rattache au départ.

OÙ VIVENT CES TROIS CORRECTIONS

Toutes trois dans le client distant, dont l’en-tête promet d’absorber les différences de protocole. Ollama et OpenAI ne sont pas affectés : sans index reçu, sans signature reçue, la charge repart inchangée. On ne fabrique rien là où rien n’a été reçu.

13.8 Supervision

Un moniteur UptimeRobot interroge /health toutes les cinq minutes.

POURQUOI /HEALTH ET NON /HEALTH/READY

Sonde	Réveille Render	Réveille Neon
/health	oui	non
/health/ready	oui	oui

Neon facture le temps de calcul de sa base. Maintenir Render éveillé sans réveiller Neon économise ce quota — et cette propriété découle d’un choix de conception antérieur : la sonde de vivacité ne touche pas la base, pour qu’une base momentanément indisponible ne fasse pas tuer une application saine.

Deux incidents ont été traités. Le 502 initial portait X-Render-Routing: no-deploy à 18 h 31, soit exactement entre l’arrêt de 18 h 30 et le démarrage de 18 h 34 : la fenêtre de redéploiement. Réel mais bénin.

Le 405 permanent était un vrai défaut : UptimeRobot interroge en HEAD — la méthode économique pour vérifier qu’un service répond sans en télécharger le corps — et la route ne déclarait que GET . Chaque vérification

était comptée comme une panne sur un service parfaitement sain, alors que les GET voisins répondaient 200 dans les mêmes secondes.

14 Journal complet des incidents

Vingt-sept incidents rencontrés, diagnostiqués et corrigés. Ils sont présentés ici avec leur symptôme observé, leur cause réelle et la preuve de la correction — car dans plus d'un tiers des cas, le symptôme désignait le mauvais coupable.

01 Intégration continue en échec depuis huit exécutions

SYMPTÔME	Le travail « Services et API » échoue sans rapport avec les commits qui le déclenchent.
CAUSE	L'image <code>postgres:16-alpine</code> ne contient pas l'extension <code>vector</code> , créée par la migration 0007. La migration s'arrêtait avant tout test.
PREUVE	<code>docker run --rm postgres:16-alpine</code> : aucun fichier d'extension <code>vector</code> . Les cinq autres extensions du schéma vivent dans <code>contrib</code> .
CORRECTION	Image <code>pgvector/pgvector:pg16</code> , celle que <code>docker-compose.prod.yml</code> utilisait déjà.

02 `alembic check` signale un écart permanent

CAUSE	La migration 0010 passait le nom complet à <code>create_check_constraint</code> , que la convention de nommage préfixait une seconde fois.
CORRECTION	Migration corrective 0011. 0010 n'est pas réécrite : elle est appliquée partout, base hébergée comprise, et une migration déjà passée se corrige par la suivante.

03 Page de statistiques vide en intégration continue

SYMPTÔME	« Pas assez de réservations » sur un jeu qui en contient 608.
CAUSE	<code>Date.getHours()</code> lit l'heure du poste. La chaîne tourne en UTC : une réservation de 9 h y devient 7 h et ne tombe dans aucune tranche.
PORTÉE	Ce n'était pas un test fragile — un navigateur réglé sur un autre fuseau affichait la même page vide.

04 Couverture du domaine à 0 % sur un module utilisé

CAUSE	<code>organisation.py</code> n'était éprouvé que depuis <code>tests/services</code> , où le travail « Domaine » ne regarde pas.
LEÇON	Un module du domaine se prouve dans le domaine.

05 422 en production, jamais en local

CAUSE	<code>fastapi==0.118.0</code> épinglé contre <code>0.141.1</code> utilisé. Sous la version épinglée, le modèle Pydantic et la dépendance de session sont pris pour des champs de corps.
PREUVE	Environnement de la chaîne reconstruit : Linux, Python 3.12, TZ=UTC, collation C. L'échec s'y reproduit et disparaît avec la bonne version.
GRAVITÉ	La production aurait servi une API cassée sur ces routes.

06 Le conteneur refuse de démarrer

SYMPTÔME	<code>exec: "/app/docker-entrypoint.sh": permission denied</code>
CAUSE	Fichier stocké en <code>100644</code> dans git. Le montage recouvre le répertoire de l'image, et sous Linux le fichier reprend son mode git. Invisible sous Windows.
CORRECTION	<code>git update-index --chmod=+x</code> , appliqué aussi aux scripts de sauvegarde et de restauration, porteurs du même défaut.

07 Parcours de bout en bout bloqué à la connexion

CAUSE	<code>.env.example</code> porte la valeur de production (5/min) et la chaîne le copie en <code>.env</code> , écrasant le plafond relevé de la pile de test.
PREUVE	Le parcours échoue à 5/minute exactement comme en CI, et passe entièrement à 100/minute.

08 Erreur technique sur un créneau déjà pris

CAUSE	Interblocage <code>40P01</code> non traduit, seule la violation d'exclusion <code>23P01</code> l'étant.
CORRECTION	Verrou consultatif par salle, pris avant la vérification et non avant l'insertion.
EFFET MESURÉ	Le fichier de concurrence passe de 6 à 14 secondes — la sérialisation fait son office.

09 Courriel de suspension jamais reçu

SYMPTÔME	Rien n'arrive, y compris dans les indésirables, et aucune erreur au journal.
CAUSE	La route de changement de statut ne programmait pas l'expédition. Le message restait dans la file du processus jusqu'au passage suivant de l'ordonnanceur — ou indéfiniment.
PREUVE	Le test porte désormais sur ce qui est <i>parti</i> et sur une file <i>vide</i> . Le regarder au travers de la file laissait passer le défaut.

10 Plan d'étage perdu après un `seed --reset`

CAUSE `floor_plans` emportée en cascade avec les bâtiments. Les fichiers survivaient, la ligne disparaissait.

PREUVE Trois fichiers dans `media/plans`, des placements journalisés à l'audit, zéro ligne dans `floor_plans`.

11 Lien mort dans la réponse de l'assistant

SYMPTÔME `` — « media » devenu nom de domaine.

CAUSE L'outil rendait l'adresse au modèle, qui la recopiait en inventant un hôte.

CORRECTION Toute clé en `_url` part dans la carte, jamais dans ce que le modèle relit. Le retrait descend dans les structures imbriquées.

12 « Aucun plan » sur une salle qui en a un

CAUSE L'écran utilisateur n'affichait que le plan d'étage. Sans lui — le cas courant — il ignorait le repère de localisation, pourtant transmis par l'API et présent pour les quinze salles.

SECOND DÉFAUT Chaque ouverture produisait un 404 sur `/floors/.../plan` : une réponse juste, que la console affiche en rouge et qui se lit comme une panne. `RoomOut` porte désormais `floor_has_plan`.

13 Statistiques identiques sur deux comptes distincts

CAUSE `private, max-age=300` sur une URL identique pour tous. Le cache navigateur est indexé par URL.

PREUVE Vérifié en base : l'un des deux comptes n'avait aucune réservation, et celle affichée appartenait à l'autre, dont il n'était pas participant.

CORRECTION `private, no-store` sur les chiffres personnels et sur l'export CSV, qui ne portait aucun en-tête.

14 Export PDF et Excel toujours en erreur

CAUSE Les deux formats étaient proposés à l'écran et refusés par la couche de données.

DÉCISION Retirés. Un bouton présent qui échoue à chaque clic est pire que pas de bouton.

15 Compteur de la page d'accueil figé cinq minutes

CAUSE `public, max-age=300` : le navigateur gardait sa copie sans rien redemander. La base était à jour, l'écran non.

CORRECTION `public, max-age=60, stale-while-revalidate=240` — la copie est servie immédiatement puis renouvelée en arrière-plan.

16 Déploiement bloqué, cause muette

CAUSE Sonde de démarrage trop discrète : quatre pistes possibles, aucune preuve.

CORRECTION Quatre améliorations successives (voir 13.4), sans jamais journaliser le mot de passe.

17 Connexion Neon refusée sur « mot de passe »

CAUSE RÉELLE Mode TLS absent du DSN. `libpq` applique `prefer` et se rabat en clair ; Neon refuse avec un message générique parlant de mot de passe.

CORRECTION `sslmode` déduit de l'environnement, surchargeable.

18 Images cassées après un redéploiement

CAUSE Disque éphémère : le fichier disparaît, la ligne en base persiste.

CORRECTION Fichiers versionnés, déposés en local contre la base de production.

19 Supervision en panne permanente sur un service sain

CAUSE UptimeRobot interroge en `HEAD` ; la route ne déclarait que `GET` et rendait 405.

PREUVE Les `GET` voisins répondaient 200 dans les mêmes secondes.

20 L'étage distant n'était jamais utilisé

CAUSE Le client refuse d'émettre sans anonymisation, et aucun des cinq points de construction du sélecteur ne lui en passait une.

PREUVE En rétablissant l'état antérieur, le sélecteur rend « Aucun fournisseur d'inférence disponible » alors que l'étage est entièrement configuré.

21 Vecteurs incohérents entre local et production

CAUSE	Base partagée, modèles d’embeddings différents. La similarité reste calculable et compare deux espaces sans rapport.
SYMPTÔME	Aucun. La recherche cesse simplement de trouver — l’échec le plus coûteux à diagnostiquer.
CORRECTION	Rôle « vecteurs » épinglé au même fournisseur des deux côtés, plus un script de réindexation.

22 Agent privé de ses outils

CAUSE	La façade Gemini omet l’ <code>index</code> des appels. Deux appels simultanés tombaient sous la clé 0, arguments concaténés et illisibles.
PREUVE	Sur un flux portant deux appels sans index, la réponse est <i>vide</i> avant le correctif et porte les deux appels après.

23 Tour interrompu au second aller-retour

SYMPTÔME	Les bons articles s’affichent, surmontés de « je n’ai pas compris la demande ».
CAUSE	Arguments d’outil renvoyés en objet là où le protocole OpenAI attend une chaîne JSON.
PREUVE	La même requête rend 200 avec une chaîne, 400 avec un objet : <code>Value is not a string</code> .

24 « Salle PERSONNE_2 » dans la liste des salles

CAUSE	L’exclusion des noms de lieux ne portait que sur le premier mot de la paire. Sur « Salle Conseil Alpha », le moteur écarte « Salle Conseil » puis masque « Conseil Alpha ».
CORRECTION	Rétrospection ajoutée en complément de l’anticipation — elle avait été essayée puis retirée comme inutile, ce qu’elle était <i>seule</i> .

25 Jetons d’anonymisation visibles dans les réponses

CAUSE	La restitution existait et n’était appelée nulle part ; l’anonymiseur était reconstruit sans mémoire à chaque appel.
DIFFICULTÉ	Le modèle diffuse « PERSON » puis « NE_1 » : traduire chaque morceau ne reconnaîtrait ni l’un ni l’autre. Un tampon retient le mot en cours et se vide en fin de flux.

26 Une carte s'affiche puis disparaît

SYMPTÔME	Le plan de la salle apparaît, puis est remplacé par une liste de salles et « j'ai cherché une salle correspondant à votre besoin ».
CAUSE	La bascule vers le repli ne regardait pas si le tour avait déjà produit quelque chose à l'écran.
DÉCISION	Le repli sait répondre depuis une page blanche, pas corriger une réponse en cours. Mieux vaut une réponse incomplète qu'une réponse qui se dédit.

27 Assistant dégradé après une dizaine de questions

CAUSE	Quota quotidien du palier gratuit : <code>limit: 20</code> requêtes par jour et par modèle, et un tour en consomme au moins deux.
DIFFICULTÉ	Le journal ne portait que le code <code>ia_fournisseur</code> , qui couvre quatre causes distinctes. Il a fallu rejouer un tour en local, en interceptant le client, pour lire le message.
CORRECTION	Modèle changé pour une variante plus généreuse, et le détail du refus joint au journal — sans jamais y joindre la conversation.

15 Écarts entre local et production

Aspect	En local	En production
Modèle de raisonnement	qwen2.5:7b via Ollama	Gemini via façade OpenAI
Modèle de vecteurs	Gemini — imposé des deux côtés pour la cohérence du corpus	
Base de données	PostgreSQL 16 en conteneur	Neon PostgreSQL 18 managé
Courriel	Boîte de développement, rien ne sort	Relais SMTP réel
Médias	Disque de travail	Image Docker, fichiers versionnés
Limiteur de connexion	100 / minute	5 / minute
Démarrage	Immédiat	≈ 50 s après quinze minutes d'inactivité

Ce qui a été vérifié en production

Vérification	Résultat
23 routes d'administration en lecture	200 avec session d'administration, 403 avec session utilisateur
Routes utilisateur (profil, salles, bâtiments, équipements, réservations, notifications, statistiques, export, FAQ, tickets, horaires, fermetures, demandes d'accès, conversations)	200
Parcours public rejoué par Playwright contre la production	4 tests verts en 13,4 s
Médias (photos, repères, plans, images de bâtiment)	200, servis en moins de 200 ms
Assistant, question sans mot commun avec sa réponse	Réponse correcte, repli: false

Les quatre limites structurelles

Aucun correctif ne les supprime, et elles sont documentées plutôt que masquées :

1. **Le réveil à froid** — le palier gratuit endort le service après quinze minutes.
2. **Le disque éphémère** — d'où le versionnement des médias.
3. **Le limiteur de connexion** — cinq par minute, qui gêne les allers-retours entre comptes lors d'une démonstration.

4. Le quota quotidien du modèle — compté par modèle et par jour.

LA PHRASE QUI RÉSUME LA CONCEPTION

La dégradation reste propre : quota épuisé, l'assistant retombe sur son moteur déterministe et continue de répondre aux intentions déclarées. **L'étage distant est un confort, pas une dépendance.**

16 Limites connues et perspectives

16.1 Ce qui n'est pas fait, et pourquoi

Export PDF et Excel. Les deux formats figuraient dans l'interface et la couche de données les refusait systématiquement. Ils ont été retirés plutôt que laissés en place. Aucun des deux ne se produit sans une dépendance nouvelle, côté serveur comme côté navigateur : c'est une décision, pas un oubli. Une voie sans dépendance existe — une vue imprimable et `window.print()` — et reste à arbitrer.

Dénombrement par l'assistant. « Combien de salles sont enregistrées » n'a pas d'outil dédié. L'assistant le dit plutôt que d'inventer un chiffre, ce qui est le comportement voulu et non un défaut.

Stockage objet externe. La bonne réponse pour une application réelle, écartée faute de temps et parce qu'elle impose de réécrire la couche de stockage.

16.2 Perspectives

Évolution	Ce qu'elle apporterait
Stockage objet externe	Supprime la contrainte de versionnement des médias et le couplage au cycle de déploiement.
Instance payante ou disque persistant	Supprime le réveil à froid et le disque éphémère.
Élargissement du catalogue d'outils	Dénombrements, agrégats, comparaisons de salles.
Vue imprimable des rapports	Export PDF sans dépendance nouvelle.
Notifications temps réel	WebSocket en complément des courriels, pour les arbitrages urgents.
Modèle de vecteurs auto-hébergé	Supprime la dépendance à un tiers pour la recherche sémantique.
Historisation des scores de recommandation	Permettrait d'évaluer la qualité des recommandations sur données réelles — la brique Data la plus naturelle à prolonger.

17 Bilan et enseignements

17.1 Ce que le projet démontre

Une garantie d'intégrité portée par la base, et non par du code applicatif : deux réservations chevauchantes ne sont pas improbables, elles sont impossibles.

Une architecture en couches dont la couche métier est pure, testable en isolation, et prouvée à 100 % de couverture de branches.

Une chaîne de tests à cinq niveaux — 1 576 tests automatisés — et une discipline de contre-épreuve qui a, au moins une fois, invalidé un correctif qui semblait bon.

Deux briques Data & IA réelles : un moteur de recommandation explicable, et un assistant outillé adossé à une recherche hybride vectorielle et lexicale, encadré par trois garde-fous.

Un déploiement complet et gratuit, mené de bout en bout, avec supervision et dégradation propre.

17.2 Trois enseignements

MESURER AVANT D’AFFIRMER

Plusieurs diagnostics initiaux se sont révélés faux : un cache de navigateur accusé à tort, un écart de nombre de salles qui n'était qu'un filtre légitime, une adresse IPv6 soupçonnée là où il manquait un mode TLS, un mot de passe déclaré faux par un serveur qui refusait en réalité une connexion non chiffrée.

À chaque fois, c'est la mesure qui a tranché — et souvent une mesure qu'il a d'abord fallu *rendre possible*, en élargissant une troncature ou en interceptant un client.

UN JOURNAL QUI DIT QU'IL A ÉCHOUÉ SANS DIRE POURQUOI COÛTE DES HEURES

Le même défaut est apparu **trois fois à trois endroits différents** : la sonde de base de données, la bascule de l'assistant, la troncature des messages d'erreur. La correction a été identique à chaque fois — joindre la cause, sans jamais joindre le secret ni la conversation.

CE QUI NE S'EXÉCUTE QUE D'UN CÔTÉ N'EST PAS TESTÉ

Six défauts vivaient dans le chemin distant de l'assistant. **Aucun n'était visible en local**, parce qu'Ollama y sert seul et qu'aucun de ces défauts n'est sur son chemin. Le déploiement n'est pas une formalité de fin de projet : c'est un environnement d'exécution à part entière, avec ses propres défauts, qui mérite ses propres vérifications.

UN PRINCIPE DE CONCEPTION QUI RÉSUME LE RESTE

Une garde qui refuse par défaut doit être **branchée** par défaut. L'anonymiseur en illustre les deux faces : la garde était juste — ne rien envoyer à un tiers sans masquer — et son absence de câblage a rendu tout un étage inatteignable pendant des semaines, sans autre signe qu'une ligne de journal. *Le défaut sûr est de masquer, pas de se taire.*

18 Annexes

18.1 Adresses

Ressource	Adresse
Page de présentation	https://smartroommanager.vercel.app/presentation
Connexion utilisateur	https://smartroommanager.vercel.app/connexion
Connexion administration	https://smartroommanager.vercel.app/admin/connexion
API	https://smartroom-api-ryya.onrender.com
Documentation OpenAPI	https://smartroom-api-ryya.onrender.com/docs
Dépôt	https://github.com/DD542/SmartRoomManager

18.2 Comptes de démonstration

Mot de passe commun : `smartroom2026` . Les comptes d'administration se connectent par `/admin/connexion` .

Compte	Rôle	Permissions
<code>d.menga@ece.fr</code>	Directeur IT, propriétaire	Toutes
<code>s.boukehila@ece.fr</code>	Directeur de site	salles, support, conflits
<code>c.nkoulou@ece.fr</code>	Référente support	support, conflits
<code>jean.dupont@edu.ece.fr</code>	Utilisateur	—
<code>alice.leroy@edu.ece.fr</code>	Utilisateur	—

18.3 Commandes utiles

```
# Pile locale
docker compose up -d db courriel
cd smartroom-api && ..\.venv\Scripts\python.exe -m uvicorn app.main:app --reload --port 8000
cd smartroom-front && npm run dev

# Base de donnees
..\venv\Scripts\python.exe -m alembic upgrade head
..\venv\Scripts\python.exe -m scripts.seed
..\venv\Scripts\python.exe -m scripts.reindexer_connaissances

# Qualite
..\venv\Scripts\python.exe -m pytest
..\venv\Scripts\python.exe -m ruff check app tests scripts
npm run test
npm run test:e2e

# Parcours public joue contre la production
E2E_BASE_URL=https://smartroommanager.vercel.app npx playwright test e2e/vitrine-animations.spec.js
```

18.4 Variables d’environnement de production

Variable	Rôle
ENVIRONMENT	production — déduit notamment le mode TLS et les garde-fous de configuration
POSTGRES_HOST/PORT/USER/PASSWORD/DB	Connexion à Neon
POSTGRES_SSLMODE	require — surcharge la déduction si nécessaire
JWT_SECRET	Signature des jetons d’accès
CORS_ORIGINS	Liste JSON des origines autorisées
REFRESH_COOKIE_SECURE	true
GOOGLE_CLIENT_ID	Identifiant public du client OAuth
MAIL_ENABLED , SMTP_* , MAIL_FROM	Relais de courriel ; MAIL_FROM doit être l’adresse authentifiée
IA_DISTANT_URL , IA_DISTANT_CLE	Étage B de l’assistant
IA_DISTANT_MODELE_RAISONNEMENT	Modèle de dialogue
IA_DISTANT_MODELE_VECTEURS	Modèle d’embeddings
IA_VECTEURS_TOUJOURS_DISTANTS	Cohérence du corpus vectorisé entre les environnements

18.5 Migrations

Révision	Objet
0001_schema_initial	Schéma initial
0002_min_advance	Délai minimal d'anticipation, configurable comme les autres règles
0003_auth_tokens	Jetons de session et de réinitialisation
0004_photo_order	Ordre des photos de salle, unicité différée jusqu'à la validation
0005_avatar	Photo de profil
0006_visuels_parc	Image de bâtiment et repère de localisation de salle
0007_rag_pgvector	Extension <code>vector</code> et table de fragments
0008_conversations	Conversations, messages et journal des tours
0009_notification_gabarit	Gabarit ayant produit une notification
0010_consigne_regles	Consigne écrite par l'administration, portée par la règle
0011_nom_contrainte_consigne	Correction du nom de contrainte doublement préfixé
0012_gabarit_suspension	Gabarit de suspension de compte
0013_gabarit_reactivation	Gabarit de réactivation de compte

18.6 Outils de l'assistant

Outil	Nature	Confirmation
rechercher_salles	lecture	non
recommander_salle	lecture	non
localiser_salle	lecture	non
consulter_disponibilite	lecture	non
lister_mes_reservations	lecture	non
obtenir_code_acces	lecture	non
consulter_regles	lecture	non
rechercher_faq	lecture	non
transférer_humain	lecture	non
créer_reservation	écriture	oui
modifier_reservation	écriture	oui
annuler_reservation	écriture	oui
créer_ticket	écriture	oui

Rapport rédigé au terme du projet. Toutes les mesures citées ont été relevées sur l'application réelle, en local ou en production, et sont reproductibles avec les commandes de l'annexe 18.3.